

Determinex

Compiler-verified AI agents.
Private, local, and measured on AMD Radeon.

Track 2 · Development & Local Deployment of Private AI Agents

github.com/DarthCeltic/Determinex · huggingface.co/darthceltic85 · AGPL-3.0-or-later

THE PROBLEM

Both options are unacceptable

Cloud coding assistants

Require shipping proprietary source to a third party, with no guarantee about retention, training use, or jurisdiction. For regulated and IP-sensitive work that is not a trade-off — it is a disqualifier.

Local assistants

Hand you one small model with nothing checking its work. A 1.5B model that is *confidently* wrong costs more review time than no assistant at all.

Determinex refuses both. It runs on your own hardware, and it never trusts a model's self-assessment — **a real compiler or a real test suite is the only judge.**

THE MECHANISM

Correctness is bought with GPU throughput

A model succeeds on some attempt with probability `p`. Sample `K` candidates, verify each against a sound oracle, keep what passes:

$$P = 1 - (1 - p)^K$$

Any `p > 0` is driven toward correct by raising K. The model supplies search; the oracle supplies truth.

So accuracy is bounded by batch throughput, not by model quality

That makes this one of the few agent architectures where a GPU property converts *directly* into a correctness property. It is the reason this project belongs on this hardware rather than merely running on it.

THE AMD ARGUMENT — MEASURED FIVE TIMES

The sixth candidate is nearly free

RUN	INSTANCE	SINGLE	K=6 AGGREGATE	GAIN	WALL CLOCK
1	vLLM Model API	30.0	166.9	5.56×	1.08×
2	Notebook, local vLLM	31.9	179.0	5.61×	1.07×
3	Notebook (on camera)	32.6	186.9	5.73×	1.05×
4	Token Factory	26.1	—	—	quota wall
5	**Live, recording**	29.5	163.0	5.61×	1.07×

Absolute rates move with instance load. **The ratio does not** — and the ratio is the claim, not any single figure.

CHOOSING K

Swept, not guessed — and the cliff is predictable

K	AGG TOK/S	VS K=1
1	28.7	—
8	220.3	1.04× time
16	413.5	1.11× time
32	**739.9**	**1.21× time**
48	849.7	1.53× time
64	512.6 ↓	collapse

vLLM announced the limit at boot

`Maximum concurrency: 61.83×` — and the measured collapse lands exactly where the KV arithmetic says it must.

A rule any Radeon owner can apply

Keep K below `kv_cache_tokens / max_model_len`. Here 506,544 / 8,192 = 61, and 64 broke.

The rule is code, not prose

`determinex_calibrate.py` derives K from the machine it is actually on. An uncalibrated rig is *told* it is uncalibrated rather than handed a confident default.

WHERE IT WORKS, AND WHERE IT DOES NOT

1,152 generations. Two models. Three regimes.

TASK	7B P	32B P	REGIME
rle	1.000	1.000	saturated — K adds nothing
gron, 5 checks	0.005	0.943	K only trims the tail
gron, 6 checks	0.000	**0.490**	**the productive middle**

At `p = 0.490` the model fails **more than half** its attempts. K=8 takes it to **99.54%** for **1.04×** the wall clock.

The outer rows are what make the middle one credible

At p=1 there is nothing to amplify — including on *our own demo task*, which this model solves 192/192. At p=0 no K helps at all, because $1-(1-0)^K = 0$. Verified search multiplies capability; it cannot manufacture it. Both boundaries are measured, not assumed.

WHERE THE PREDICTION MISSED

The equation was handed the wrong K

Measured `p = 0.0625`, K=8. The equation predicted **0.403**. Six real searches returned **0.167**.
So we measured p at every rung of the temperature ladder, on the GPU:

RUNG	T=0.0	0.2	0.4	0.6	0.8	1.0
p	0.00	0.00	0.00	0.00	**0.10**	**0.10**

Every draw at t ≤ 0.6 has **p = 0.00**. Four of the eight samples could not pass, so a **nominal K of 8 was an effective K of 4** — and the corrected prediction, 0.185–0.344, brackets the 0.167 observed.

Reported because it missed, not despite it

The per-rung hypothesis was *wrong* too, and that is in the document as well. A section that shows only the explanation that worked is indistinguishable from one that went looking for a flattering number.

APPLICATION VALUE

A real repository, repaired on the GPU

`mwaskom__seaborn-3010` from SWE-bench Lite — a real reported issue, with the project's own test as the judge.

The blame is earned

`CODE` — not ENVIRONMENT, not TEST — and `proven_slop: 0`. It did not take the easy exit of declaring the failing test wrong.

The oracle is theirs, not ours

The repository's own pytest, run with its own interpreter. Nothing Determinex wrote decides whether this passed.

There is a control

SWE-bench's official gold patch also passes this oracle. Without that control, a pass proves much less.

27 seconds, end to end

```
p = np.polyfit(x.dropna(), y.dropna(), self.order)
```

BOUNDARIES

What this does *not* do

The design rule is that a check must never report an outcome it did not establish. Applied to a submission, the limits belong in it at the same volume as the results.

- → **Verified search cannot manufacture capability.** Measured 0/192 at p=0. No K rescues a model that cannot do the task.
- → **A sound-but-wrong oracle is not detectable from inside the system.**
- → **Our own demo task is trivial** — 192/192 one-shot. Amplification contributes nothing there, and we would rather say so than let it be inferred.
- → **Path B is documented, not measured.** No physical Radeon card ran it.
- → **The p/K validation is consistent with the law, not strong evidence for it** — 10 draws per rung, 6 trials.
- → **Three execution sites are trusted-by-design** rather than sandboxed, and left flagged rather than silently exempted.

VERIFY IT YOURSELF

Four commands, about two minutes

```
git clone https://github.com/DarthCeltic/Determinex && cd Determinex
pip install -r scripts/requirements.txt
export DETERMINEX_VLLM_BASE_URL=http://127.0.0.1:8000/v1
python scripts/determinex_build_from_idea.py --idea idea.md --provider vllm --k 32
```

Verified from a clean clone: it returns **SOLVED**, three synthesized checks, oracle-verified.

Then point it at something that cannot be verified — `improve the code quality of my project` — and it **refuses**: *"no example or typeable invariant could be derived… that cannot verify behaviour."* **That refusal is the product.**

IN ONE LINE

A local agent whose accuracy is bounded by what an AMD Radeon can verify per second — and which refuses to claim anything it cannot prove.

5.61×
throughput at 1.07× wall clock, confirmed on five instances

99.54%
solve rate from a coin-flip model, for 4% more time

0
claims without a passing oracle result and a proof

github.com/DarthCeltic/Determinex · huggingface.co/darthceltic85 · AGPL-3.0-or-later